



PERTEMUAN 4 TRANSFORMASI GEOMETRI 2D

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu memahami dan mengimplementasikan konsep transformasi geometri 2D yang meliputi translasi, rotasi, refleksi, *shear* dan *scaling*.

Pokok Bahasan:

1. Apa yang dimaksud dengan transformasi 2 dimensi pada objek grafik
2. Proses transformasi windows-viewport serta komputasinya.
3. Proses transformasi dasar dan melakukan proses komputasi transformasi dasar (translasi, rotasi, refleksi, shear dan scaling).
4. Implementasi translasi, rotasi, refleksi, shear dan scaling pada program dengan Java.
5. Soal Latihan

4.1 Pengertian Transformasi

Grafika komputer merupakan bidang yang menarik minat banyak orang. Salah satu sub bagian dari grafika komputer adalah **pemodelan objek (*object modelling*)**. Dalam pemodelan objek dua dimensi (2D), didapati berbagai objek dapat dimodelkan menurut kondisi tertentu, objek yang dimodelkan itu perlu **dimodifikasi**. Pemodifikasian objek ini dapat dilakukan dengan melakukan berbagai operasi fungsi atau operasi transformasi geometri. Transformasi ini dapat berupa transformasi dasar ataupun gabungan dari berbagai transformasi geometri. Transformasi geometri digunakan untuk memberikan metode metode **perubahan bentuk** dan **posisi dari sebuah obyek**. Sehingga transformasi merupakan alat yang paling mendasar yang digunakan untuk grafika komputer. Transformasi membantu menyederhanakan tugas-tugas dari pemodelan geometri (*geometric modeling*), animasi, dan rendering.

Contoh transformasi geometri adalah **translasi, penskalaan, putaran (rotasi), balikan, *shearing* dan gabungan**. Transformasi ini dikenal dengan transformasi ***affine***. Pada dasarnya, transformasi ini adalah **memindahkan objek tanpa merusak bentuk**.

Tujuan transformasi adalah :

1. **Merubah atau menyesuaikan komposisi pemandangan**
2. **Memudahkan membuat objek yang simetris**
3. **Melihat objek dari sudut pandang yang berbeda**
4. **Memindahkan satu atau beberapa objek dari satu tempat ke tempat lain**, ini biasa dipakai untuk animasi komputer.

4.1.1 Translasi (Pergeseran)

Transformasi translasi merupakan suatu operasi yang menyebabkan perpindahan objek 2D dari satu tempat ke tempat yang lain. Perubahan ini berlaku dalam arah yang sejajar dengan **sumbu X dan sumbu Y**. Translasi dilakukan dengan penambahan translasi pada suatu titik koordinat dengan **translation vector**, yaitu **(*tx*, *ty*)**, dimana ***tx* adalah translasi menurut sumbu x** dan ***ty* adalah translasi menurut sumbu y**. Koordinat baru titik yang ditranslasi dapat diperoleh dengan menggunakan rumus :

$$\begin{aligned} x' &= x + tx & (x, y) &\rightarrow \text{titik asal sebelum translasi} \\ y' &= y + ty & (x', y') &\rightarrow \text{titik baru hasil translasi} \end{aligned}$$



Translasi adalah transformasi dengan bentuk yang tetap, memindahkan objek apa adanya. Setiap titik dari objek akan ditranslasikan dengan besaran yang sama.

Dalam operasi translasi, setiap titik pada suatu entitas yang ditranslasi bergerak dalam jarak yang sama. Pergerakan tersebut dapat berlaku dalam arah sumbu X saja, atau dalam arah sumbu Y saja atau keduanya.

Translasi juga berlaku pada garis, objek atau gabungan objek 2D yang lain. Untuk hal ini, setiap titik pada garis atau objek yang ditranslasi dalam arah x dan y masing-masing sebesar t_x, t_y .

Contoh:

Untuk menggambarkan translasi suatu objek berupa segitiga dengan koordinat A(10,10) B(30,10) dan C(10,30) dengan $t_x, t_y(10,20)$, tentukan koordinat yang barunya!

Jawab:

$$\begin{aligned} \text{A : } & x' = 10 + 10 = 20 \\ & y' = 10 + 20 = 30 \\ & A' = (20, 30) \\ \text{B : } & x' = 30 + 10 = 40 \\ & y' = 10 + 20 = 30 \\ & B' = (40, 30) \\ \text{C : } & x' = 10 + 10 = 20 \\ & y' = 30 + 20 = 50 \\ & C' = (20, 50) \end{aligned}$$

4.1.2 Scalling (Penskalaan)

Penskalaan adalah suatu operasi yang membuat suatu objek berubah ukurannya baik menjadi mengecil ataupun membesar secara seragam atau tidak seragam tergantung pada faktor penskalaan (*scaling factor*) yaitu (s_x, s_y) yang diberikan. s_x adalah faktor penskalaan menurut sumbu x dan s_y faktor penskalaan menurut sumbu y. Koordinat baru diperoleh dengan:

$$\begin{aligned} x' &= x * s_x & (x, y) &\rightarrow \text{titik asal sebelum diskala} \\ y' &= y * s_y & (x', y') &\rightarrow \text{titik setelah diskala} \end{aligned}$$

Nilai lebih dari 1 menyebabkan objek diperbesar, sebaliknya bila nilai lebih kecil dari 1, maka objek akan diperkecil. Bila (s_x, s_y) mempunyai nilai yang sama, maka skala disebut dengan *uniform scaling*.

Contoh:

Untuk menggambarkan skala suatu objek berupa segitiga dengan koordinat A(10,10) B(30,10) dan C(10,30) dengan $(s_x, s_y) (3,2)$, tentukan koordinat yang barunya!

Jawab:

$$\begin{aligned} \text{A : } & x' = 10 * 3 = 30 \\ & y' = 10 * 2 = 20 \\ & A' = (30, 20) \\ \text{B : } & x' = 30 * 3 = 90 \\ & y' = 10 * 2 = 20 \\ & B' = (90, 20) \\ \text{C : } & x' = 10 * 3 = 30 \\ & y' = 30 * 2 = 60 \\ & C' = (30, 60) \end{aligned}$$

4.1.3 Rotasi (Perputaran)

Putaran adalah suatu operasi yang menyebabkan objek bergerak berputar pada titik pusat atau pada sumbu putar yang dipilih berdasarkan sudut putaran tertentu. Untuk melakukan rotasi diperlukan sudut rotasi dan pivot point (xp,yp) dimana objek akan dirotasi.

Putaran biasa dilakukan pada satu titik terhadap sesuatu sumbu tertentu misalnya sumbu x, sumbu y atau garis tertentu yang sejajar dengan sembarang sumbu tersebut. Titik acuan putaran dapat sembarang baik di titik pusat atau pada titik yang lain. Aturan dalam geometri, jika putaran dilakukan searah jarum jam, maka nilai sudutnya adalah negatif. Sebaliknya, jika dilakukan berlawanan arah dengan arah jarum jam nilai sudutnya adalah positif.

Rotasi dapat dinyatakan dengan:

$$\begin{aligned}x' &= r \cos(\phi + \theta) = r \cos \phi \cos \theta - r \sin \phi \sin \theta \\y' &= r \sin(\phi + \theta) = r \cos \phi \sin \theta + r \sin \phi \cos \theta\end{aligned}$$

sedangkan diketahui :

$$x = r \cos \phi, y = r \sin \phi$$

lakukan substitusi, maka :

$$\begin{aligned}x' &= x \cos \phi - y \sin \theta \\y' &= x \sin \phi + y \cos \theta\end{aligned}$$

Matriks rotasi dinyatakan dengan :

$$\begin{aligned}P' &= R.P \\R &= \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}\end{aligned}$$

Rotasi suatu titik terhadap pivot point (xp,yp) :

$$\begin{aligned}x' &= xp + (x - xp) \cos \theta - (y - yp) \sin \theta \\y' &= yp + (x - xp) \sin \theta + (y - yp) \cos \theta\end{aligned}$$

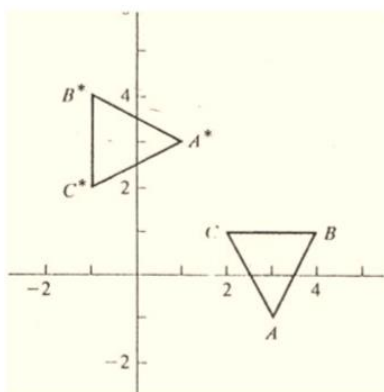
Contoh:

1. Diketahui koordinat titik yang membentuk segitiga {(3, -1), (4, 1), (2, 1)}. Gambarkan objek tersebut kemudian gambarkan pula objek baru yang merupakan transformasi rotasi objek lama sebesar 90° CCW dengan pusat rotasi (0,0).

Jawab:

Matriks transformasi umum adalah:

$$\begin{aligned}[T_{90}] &= \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} & [T_{180}] &= \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \\[T_{270}] &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} & [T_{360}] &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\end{aligned}$$



Maka dengan mengalikan titik-titik segitiga tersebut dengan matriks transformasi yang sesuai, maka akan diperoleh hasil yang digambarkan sebagai berikut:

Gambar 4.1 Ilustrasi Rotasi



2. Untuk menggambarkan rotasi suatu objek berupa segitiga dengan koordinat A(10,10), B(30,10) dan C(10,30) dengan sudut rotasi 30° terhadap titik pusat cartesian (10,10), dilakukan dengan menghitung koordinat hasil rotasi tiap titik satu demi satu.

Jawab:

Titik A

$$\begin{aligned} x' &= x_p + (x - x_p) \cos \theta - (y - y_p) \sin \theta \\ &= 10 + (10 - 10) \cdot 0.9 - (10 - 10) \cdot 0.5 \\ &= 10 \end{aligned}$$

$$\begin{aligned} y' &= y_p + (x - x_p) \sin \theta + (y - y_p) \cos \theta \\ &= 10 + (10 - 10) \cdot 0.5 - (10 - 10) \cdot 0.9 \\ &= 10 \end{aligned}$$

Titik A'(10,10)

Titik B

$$\begin{aligned} x' &= x_p + (x - x_p) \cos \theta - (y - y_p) \sin \theta \\ &= 10 + (30 - 10) \cdot 0.9 - (10 - 10) \cdot 0.5 \\ &= 28 \end{aligned}$$

$$\begin{aligned} y' &= y_p + (x - x_p) \sin \theta + (y - y_p) \cos \theta \\ &= 10 + (30 - 10) \cdot 0.5 - (10 - 10) \cdot 0.9 \\ &= 20 \end{aligned}$$

Titik B'(28,20)

Titik C

$$\begin{aligned} x' &= x_p + (x - x_p) \cos \theta - (y - y_p) \sin \theta \\ &= 10 + (10 - 10) \cdot 0.9 - (30 - 10) \cdot 0.5 \\ &= 0 \end{aligned}$$

$$\begin{aligned} y' &= y_p + (x - x_p) \sin \theta + (y - y_p) \cos \theta \\ &= 10 + (10 - 10) \cdot 0.5 - (30 - 10) \cdot 0.9 \\ &= 28 \end{aligned}$$

Titik C'(0,28)

4.1.4 Refleksi (Pencerminan)

Refleksi adalah transformasi yang membuat *mirror* (pencerminan) dari *image* suatu objek. *Image mirror* untuk refleksi 2D dibuat *relatif terhadap sumbu* dari refleksi dengan memutar 180° terhadap refleksi. Sumbu refleksi dapat dipilih pada bidang x,y. Refleksi terhadap garis $y=0$, yaitu sumbu x dinyatakan dengan matriks.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$$

Transformasi membuat nilai x sama tetapi membalikan nilai y berlawanan dengan posisi koordinat. Langkah :

- Objek diangkat
- Putar 180° terhadap sumbu x dalam 3D
- Letakkan pada bidang x,y dengan posisi berlawanan
- Refleksi terhadap sumbu y membalikan koordinat dengan nilai y tetap.

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

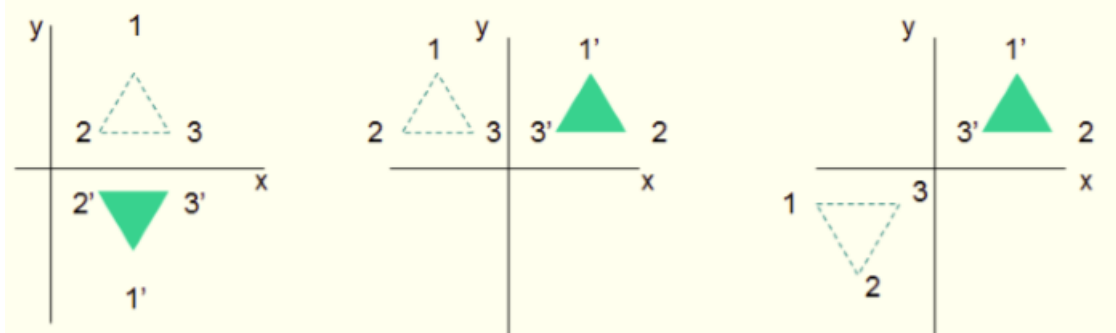
Refleksi terhadap sumbu x dan y sekaligus dilakukan dengan refleksi pada sumbu x terlebih dahulu, hasilnya kemudian direfleksikan terhadap sumbu y . Transformasi ini dinyatakan dengan :

$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$$

Refleksi ini sama dengan rotasi 180° pada bidang xy dengan koordinat menggunakan titik pusat koordinat sebagai *pivot point*. Refleksi suatu objek terhadap garis $y=x$ dinyatakan dengan bentuk matriks.

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Ilustrasi proses refleksi pada sumbu-sumbu utama digambarkan pada gambar berikut:



Gambar 4.2 Ilustrasi Refleksi

Matriks dapat diturunkan dengan menggabungkan suatu sekuen rotasi dari sumbu koordinat merefleksikan matriks. Pertama-tama dilakukan rotasi searah jarum jam dengan sudut 45° yang memutar garis $y=x$ terhadap sumbu x . Kemudian objek direfleksikan terhadap sumbu y , setelah itu objek dan garis $y=x$ dirotasi kembali ke arah posisi semula berlawanan arah dengan jarum jam dengan sudut rotasi 90° .

Untuk mendapatkan refleksi terhadap garis $y=-x$ dapat dilakukan dengan tahap :

- Rotasi 45° searah jarum jam
- Refleksi terhadap axis y
- Rotasi 90° berlawanan arah dengan jarum jam

Dinyatakan dengan bentuk matriks:

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \end{pmatrix}$$

Refleksi terhadap garis $y=mx+b$ pada bidang xy merupakan kombinasi transformasi translasi – rotasi – refleksi.

- Lakukan translasi mencapai titik perpotongan koordinat
- Rotasi ke salah satu sumbu
- Refleksi objek menurut sumbu tersebut
- Objek dan garis dirotasi sehingga mencapai sumbu lainnya.

4.1.5 Shear (Distorsi)

Shear adalah bentuk transformasi yang membuat distorsi dari bentuk suatu objek, seperti menggeser sisi tertentu. Terdapat dua macam *shear*, yaitu *shear* terhadap sumbu x dan *shear* terhadap sumbu y .

1. *Shear* terhadap sumbu x

$$\begin{pmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

Dengan koordinat transformasi

$$x' = x + sh_x \cdot y \quad y' = y$$

Parameter **sh_x** dinyatakan dengan sembarang bilangan. Posisi kemudian digeser menurut arah *horizontal*.

2. Shear terhadap sumbu y

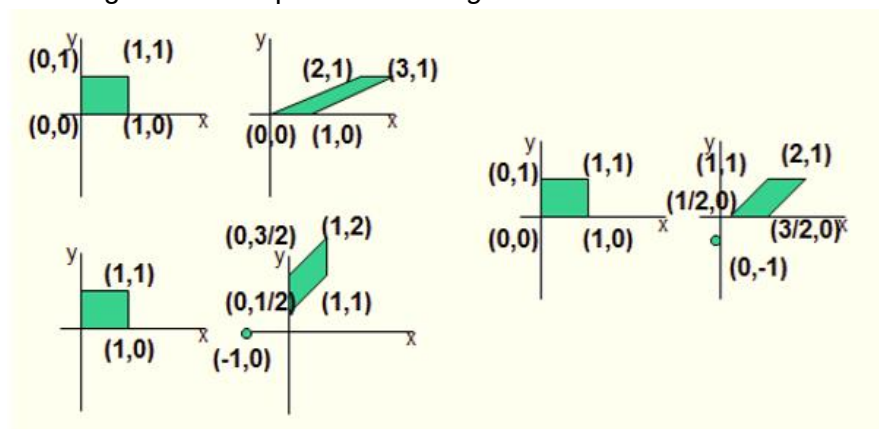
$$\begin{pmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \end{pmatrix}$$

Dengan koordinat transformasi

$$x' = x \quad y' = sh_y \cdot x + y$$

Parameter **sh_y** dinyatakan dengan sembarang bilangan. Posisi koordinat kemudian menurut arah *vertikal*.

Gambar di bawah mengilustrasikan proses *shearing*.



Gambar 4.3 Ilustrasi Proses Shearing

4.2 Sistem Koordinat Homogen 2D

Dari berbagai bentuk matrik penyajian, terlihat bahwa hanya transformasi translasi saja yang belum bisa dinyatakan sebagai matrik penyajian, karena diperlukan operasi perkalian dan penjumlahan, sedangkan pada jenis transformasi yang lain cukup diperlukan operasi perkalian matriks saja, sehingga perlu dicari suatu cara agar translasi pun juga bisa dinyatakan dalam operasi perkalian matriks. Hal ini dapat dilakukan dengan menggunakan sistem koordinat homogen.

Sistem koordinat homogen adalah sistem koordinat yang mempunyai satu dimensi lebih tinggi dari sistem koordinat yang ditinjau. Sebagai contoh, sistem koordinat homogen dari sistem koordinat dua dimensi adalah sistem koordinat 3 dimensi dengan cara menentukan salah satu sumbunya sebagai suatu konstanta. Dengan menggunakan sistem koordinat homogen, persamaan umum transformasi titik $A(x,y)$ menjadi $A'(x',y')$ dapat ditulis sebagai:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & c & t_x \\ b & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Dari persamaan tersebut, maka masing-masing transformasi diatas bisa dituliskan sebagai berikut:

- Matrik penyajian Translasi :

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Sehingga :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Matrik penyajian Scalling :

$$S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Sehingga :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Matrik penyajian Rotasi :

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Sehingga :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Contoh:

Tentukan posisi dari segitiga ABC yang dibentuk oleh titik-titik A(10,2), B(10,8) dan C(3,2) jika dilakukan transformasi berikut :

- Translasi kearah sumbu x = 4, kearah sumbu y = -2
- Scalling dengan skala kearah sumbu x = 2, kearah sumbu y = -2
- Diputar 90° berlawanan jarum jam

Jawab:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 4 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 10 & 10 & 3 \\ 2 & 8 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 14 & 14 & 7 \\ 0 & 6 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

a)

diperoleh A'(14,0), B'(14,6) dan C'(7,0)

$$b) \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 10 & 10 & 3 \\ 2 & 8 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 20 & 20 & 6 \\ -4 & -16 & -4 \\ 1 & 1 & 1 \end{bmatrix}$$

diperoleh A'(20, -4), B'(20, -16) dan C'(6, -4)

$$c) \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 10 & 10 & 3 \\ 2 & 8 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} -2 & -8 & -2 \\ 10 & 10 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

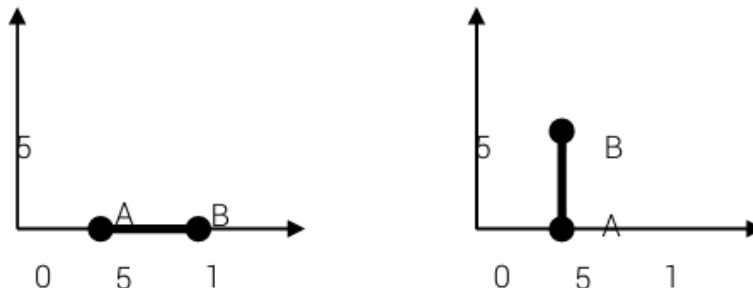
diperoleh A'(-2, 10), B'(-8, 10) dan C'(-2, 3)

4.3 Komposisi Matriks Transformasi 2D

Dengan menggunakan matrik penyajian, kita bisa menyusun transformasi secara berurutan yang biasa disebut sebagai komposisi matrik tranformasi, yaitu dengan cara menghitung perkalian matrik penyajian secara berurutan. Karena perkalian matrik tidak bersifat komutatif, maka urut-urutan perkalian matrik harus diperhatikan (tidak boleh kebalik) karena menunjukkan urut-urutan transformasi yang dilakukan.

Contoh:

Sebuah segmen garis AB dimana A(0,5) dan B(10,0) akan dirotasikan 90° terhadap titik A. Bagaimana proses ini dilakukan? Apakah langsung dilakukan proses rotasi 90° ?



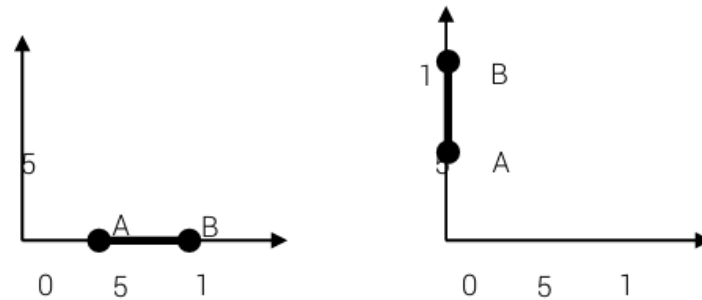
Jawab:

Jika langsung dirotasikan 90° hasilnya sebagai berikut:

$$\begin{bmatrix} x'_x \\ y'_y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(90) & -\sin(90) & 0 \\ \sin(90) & \cos(90) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5 & 10 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x'_x \\ y'_y \\ 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5 & 10 \\ 0 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 5 & 10 \\ 1 & 1 \end{bmatrix}$$

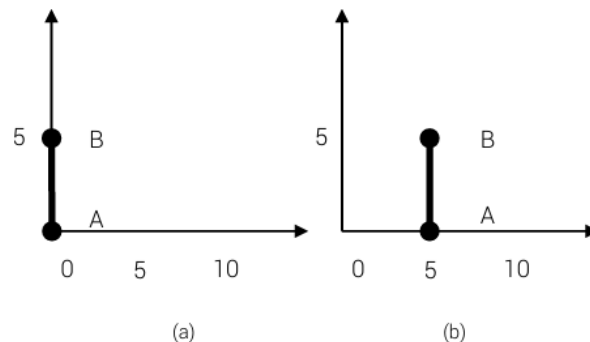
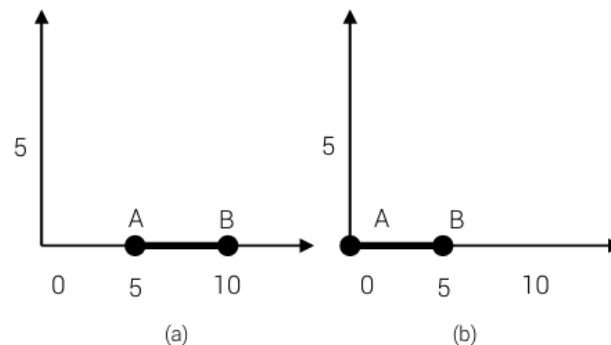
Diperoleh titik hasil transformasi : A'(0,5) dan B'(0,10)



Tentu saja hasil ini tidak seperti yang kita harapkan (hasilnya salah).

Cara yang benar adalah menggunakan komposisi transformasi, yaitu:

- Translasikan segmen garis tersebut hingga titik A (sebagai pusat rotasi) berada di titik asal (0,0).
- Kemudian rotasikan 90° terhadap titik asal, dan terakhir
- Translasikan titik A sehingga posisinya berada di tempat semula.



Secara matematis dapat ditulis sebagai berikut : $AB' = T(5) R(90) T(-5) AB$

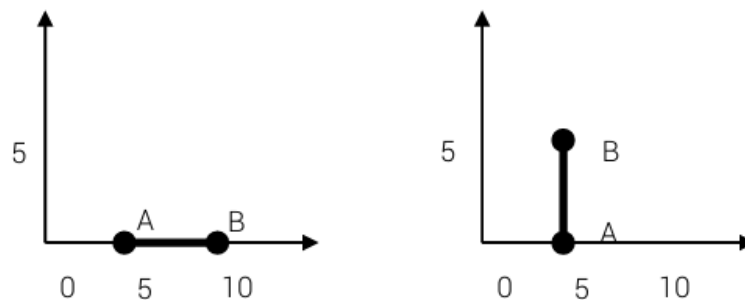
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} \cos(90) & -\sin(90) & 0 \\ \sin(90) & \cos(90) & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} \cos(90) & -\sin(90) & 0 \\ \sin(90) & \cos(90) & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 5 & 10 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 5 & 10 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 5 & 5 \\ 0 & 5 \\ 1 & 1 \end{bmatrix}$$

Diperoleh titik hasil transformasi : A'(5,0) dan B'(5,5)



Contoh:

Tentukan posisi dari segitiga ABC yang dibentuk oleh titik-titik A(5,5), B(10,5) dan C(8,10) jika dilakukan komposisi transformasi berikut: penggeseran pada $\begin{bmatrix} 6 \\ 6 \end{bmatrix}$, dilanjutkan dengan rotasi 90° berlawanan jarum jam, kemudian diakhiri dengan penskalaan dengan faktor skala $\begin{bmatrix} 3 \\ 3 \end{bmatrix}$ terhadap titik pusat P(0,0).

Jawab:

Dalam hal ini kita harus menghitung matrik komposisi berikut :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = S * R * T * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Proses dilakukan terhadap operasi translasi terlebih dahulu, kemudian dirotasi dan dilanjutkan dengan penskalaan sehingga menjadi:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} \cos(90) & -\sin(90) & 0 \\ \sin(90) & \cos(90) & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 6 \\ 0 & 1 & 6 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 5 & 10 & 8 \\ 5 & 5 & 10 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -33 & -33 & -48 \\ 33 & 48 & 42 \\ 1 & 1 & 1 \end{bmatrix}$$

Diperoleh titik hasil transformasi: A'(-33, 33), B'(-33, 48) dan C'(-48, 42)

4.4 Implementasi translasi, rotasi, refleksi, shear dan scaling pada program dengan Java

A. Kode lengkap untuk Translasi objek 2D

```
// Original polygon (rectangle)
float[][] shape = {
    {50, 50},
    {150, 50},
    {150, 100},
    {50, 100}
};

// Translation factors
float tx = 100; // translate 100 pixels right
float ty = 50;  // translate 50 pixels down

void setup() {
    size(300, 200);
    background(255);

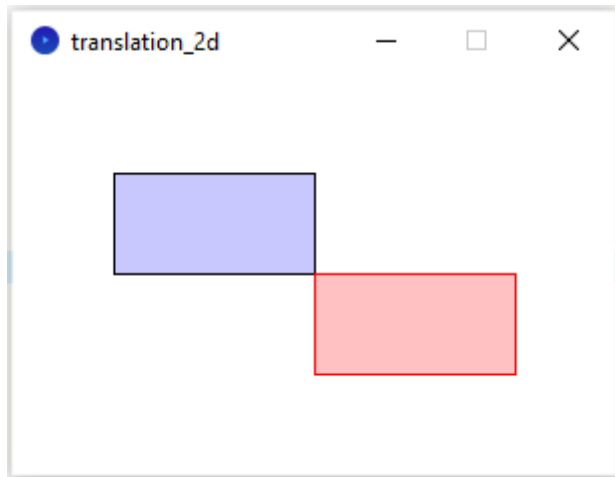
    // Draw original shape
    stroke(0);
    fill(200, 200, 255);
    drawShape(shape);

    // Translate shape using x' = x + tx, y' = y + ty
    float[][] translatedShape = translateShape(shape, tx, ty);

    // Draw translated shape
    stroke(255, 0, 0);
    fill(255, 150, 150, 150);
    drawShape(translatedShape);
}

void drawShape(float[][] points) {
    beginShape();
    for (int i = 0; i < points.length; i++) {
        vertex(points[i][0], points[i][1]);
    }
    endShape(CLOSE);
}

float[][] translateShape(float[][] points, float tx, float ty) {
    float[][] translated = new float[points.length][2];
    for (int i = 0; i < points.length; i++) {
        translated[i][0] = points[i][0] + tx; // x'
        translated[i][1] = points[i][1] + ty; // y'
    }
    return translated;
}
```



B. Kode lengkap untuk penskalaan objek 2D

```
float[][] shape = {
    {50, 50},
    {150, 50},
    {150, 100},
    {50, 100}
};

float sx = 2.0; // Scaling factor for X
float sy = 1.5; // Scaling factor for Y

void setup() {
    size(400, 300);
    background(255);

    // Draw original shape
    stroke(0);
    fill(200, 200, 255);
    drawShape(shape);

    // Apply scaling
    float[][] scaledShape = scaleShape(shape, sx, sy);

    // Draw scaled shape
    stroke(255, 0, 0);
    fill(255, 150, 150, 150);
    drawShape(scaledShape);
}

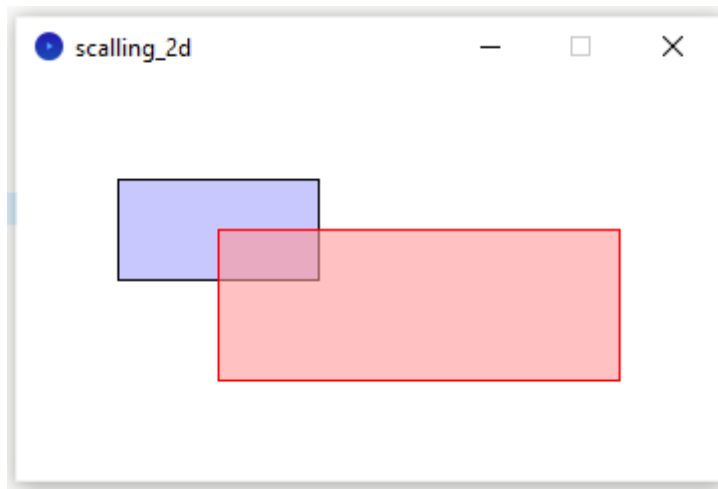
void drawShape(float[][] points) {
    beginShape();
    for (int i = 0; i < points.length; i++) {
        vertex(points[i][0], points[i][1]);
    }
    endShape(CLOSE);
}

float[][] scaleShape(float[][] points, float sx, float sy) {
    float[][] scaled = new float[points.length][2];
    for (int i = 0; i < points.length; i++) {
        scaled[i][0] = points[i][0] * sx; // x'
        scaled[i][1] = points[i][1] * sy; // y'
    }
}
```

```

    }
    return scaled;
  }

```



C. Kode lengkap untuk rotasi objek 2D

```

float[][] shape = {
    {100, 100},
    {150, 100},
    {150, 150},
    {100, 150}
};

float angle = radians(45); // rotate 45 degrees

void setup() {
    size(300, 200);
    background(255);

    // Draw original shape
    stroke(0);
    fill(200, 200, 255);
    drawShape(shape);

    // Calculate center of polygon
    float[] center = getCenter(shape);

    // Rotate shape
    float[][] rotatedShape = rotateShape(shape, angle, center[0],
    center[1]);

    // Draw rotated shape
    stroke(255, 0, 0);
    fill(255, 150, 150, 150);
    drawShape(rotatedShape);
}

void drawShape(float[][] points) {
    beginShape();
    for (int i = 0; i < points.length; i++) {
        vertex(points[i][0], points[i][1]);
    }
    endShape(CLOSE);
}

```

```

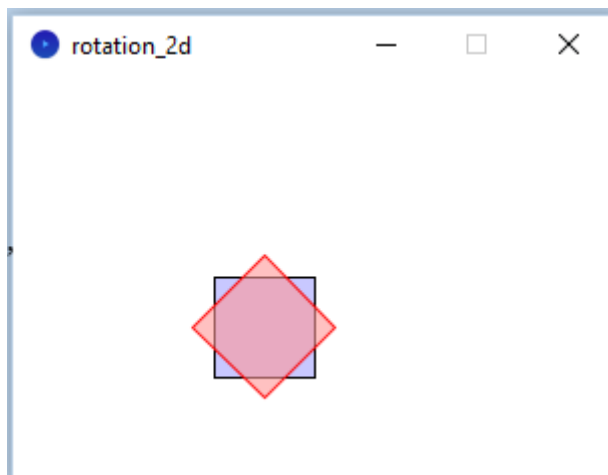
}

float[][] rotateShape(float[][] points, float theta, float cx, float
cy) {
    float[][] rotated = new float[points.length][2];
    for (int i = 0; i < points.length; i++) {
        float x = points[i][0];
        float y = points[i][1];

        rotated[i][0] = cos(theta) * (x - cx) - sin(theta) * (y - cy) +
cx;
        rotated[i][1] = sin(theta) * (x - cx) + cos(theta) * (y - cy) +
cy;
    }
    return rotated;
}

float[] getCenter(float[][] points) {
    float sumX = 0, sumY = 0;
    for (int i = 0; i < points.length; i++) {
        sumX += points[i][0];
        sumY += points[i][1];
    }
    return new float[]{sumX / points.length, sumY / points.length};
}

```



D. Kode lengkap untuk refleksi objek 2D

```

// Triangle points (x, y)
float[][] triangle = {
    {200, 100},
    {250, 200},
    {150, 200}
};

void setup() {
    size(600, 400);
    background(255);
    strokeWeight(2);

    // Original triangle
    stroke(0);
    fill(200, 200, 255);

```

```

drawShape(triangle);

// Reflect across X-axis
float[][] reflectedX = reflectX(triangle);
stroke(255, 0, 0);
fill(255, 200, 200, 150);
drawShape(reflectedX);

// Reflect across Y-axis
float[][] reflectedY = reflectY(triangle);
stroke(0, 255, 0);
fill(200, 255, 200, 150);
drawShape(reflectedY);

// Reflect across origin
float[][] reflectedOrigin = reflectOrigin(triangle);
stroke(0, 0, 255);
fill(200, 200, 255, 150);
drawShape(reflectedOrigin);

// Reflect across y = x
float[][] reflectedYX = reflectYEqualsX(triangle);
stroke(255, 0, 255);
fill(255, 255, 150, 150);
drawShape(reflectedYX);
}

void drawShape(float[][] points) {
  beginShape();
  for (int i = 0; i < points.length; i++) {
    vertex(points[i][0], points[i][1]);
  }
  endShape(CLOSE);
}

// Reflect over X-axis
float[][] reflectX(float[][] points) {
  float[][] result = new float[points.length][2];
  for (int i = 0; i < points.length; i++) {
    result[i][0] = points[i][0];
    result[i][1] = height - points[i][1];
  }
  return result;
}

// Reflect over Y-axis
float[][] reflectY(float[][] points) {
  float[][] result = new float[points.length][2];
  for (int i = 0; i < points.length; i++) {
    result[i][0] = width - points[i][0];
    result[i][1] = points[i][1];
  }
  return result;
}

// Reflect over origin
float[][] reflectOrigin(float[][] points) {
  float[][] result = new float[points.length][2];
  for (int i = 0; i < points.length; i++) {
    result[i][0] = width - points[i][0];

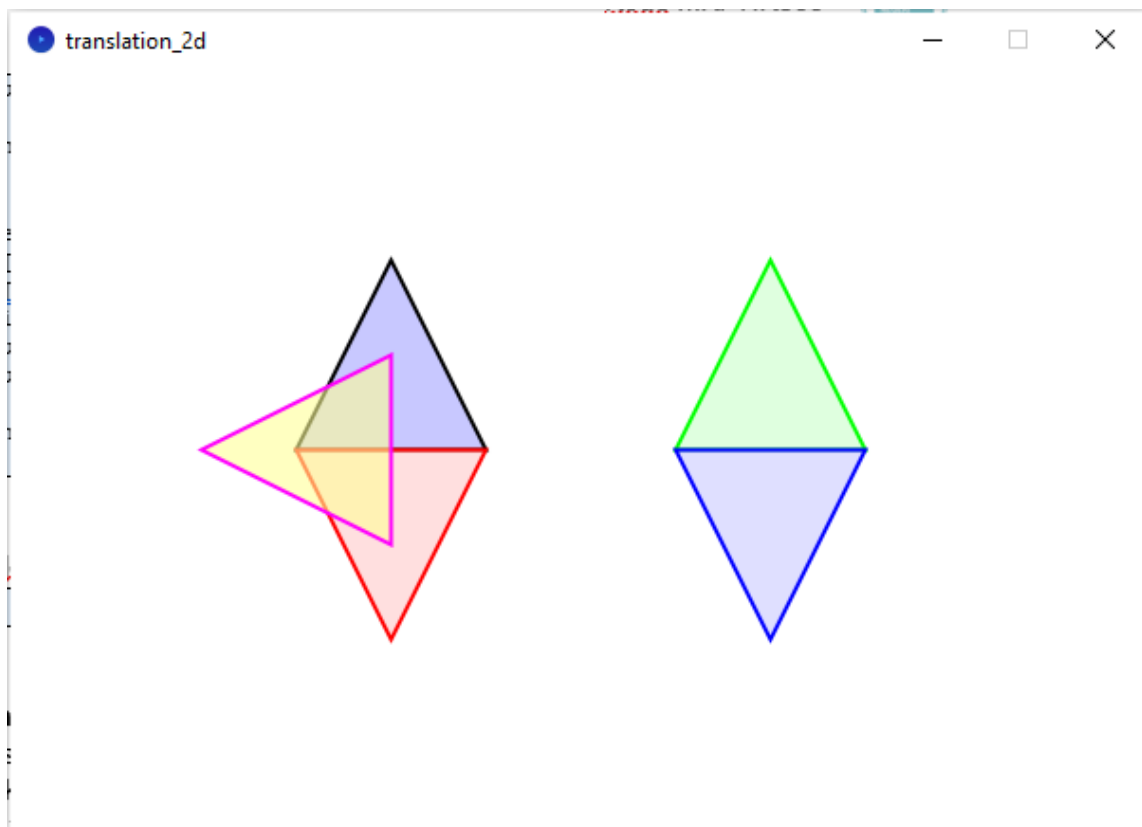
```

```

    result[i][1] = height - points[i][1];
}
return result;
}

// Reflect over line y = x
float[][] reflectYEqualsX(float[][] points) {
    float[][] result = new float[points.length][2];
    for (int i = 0; i < points.length; i++) {
        result[i][0] = points[i][1];
        result[i][1] = points[i][0];
    }
    return result;
}

```



E. Kode lengkap untuk shear objek 2D

```

float[][] rectangle = {
    {100, 100},
    {200, 100},
    {200, 200},
    {100, 200}
};

float shx = 0.5; // Shear in x-direction
float shy = 0.3; // Shear in y-direction

void setup() {
    size(600, 400);
    background(255);
    strokeWeight(2);
    textSize(14);
}

```



```

// Original rectangle
stroke(0);
fill(200, 200, 255);
drawShape(rectangle);
fill(0);
text("Original", rectangle[0][0], rectangle[0][1] - 10);

// Sheared in X
float[][] shearedX = shearX(rectangle, shx);
stroke(255, 0, 0);
fill(255, 200, 200, 150);
drawShape(shearedX);
fill(0);
text("Sheared X", shearedX[0][0], shearedX[0][1] - 10);

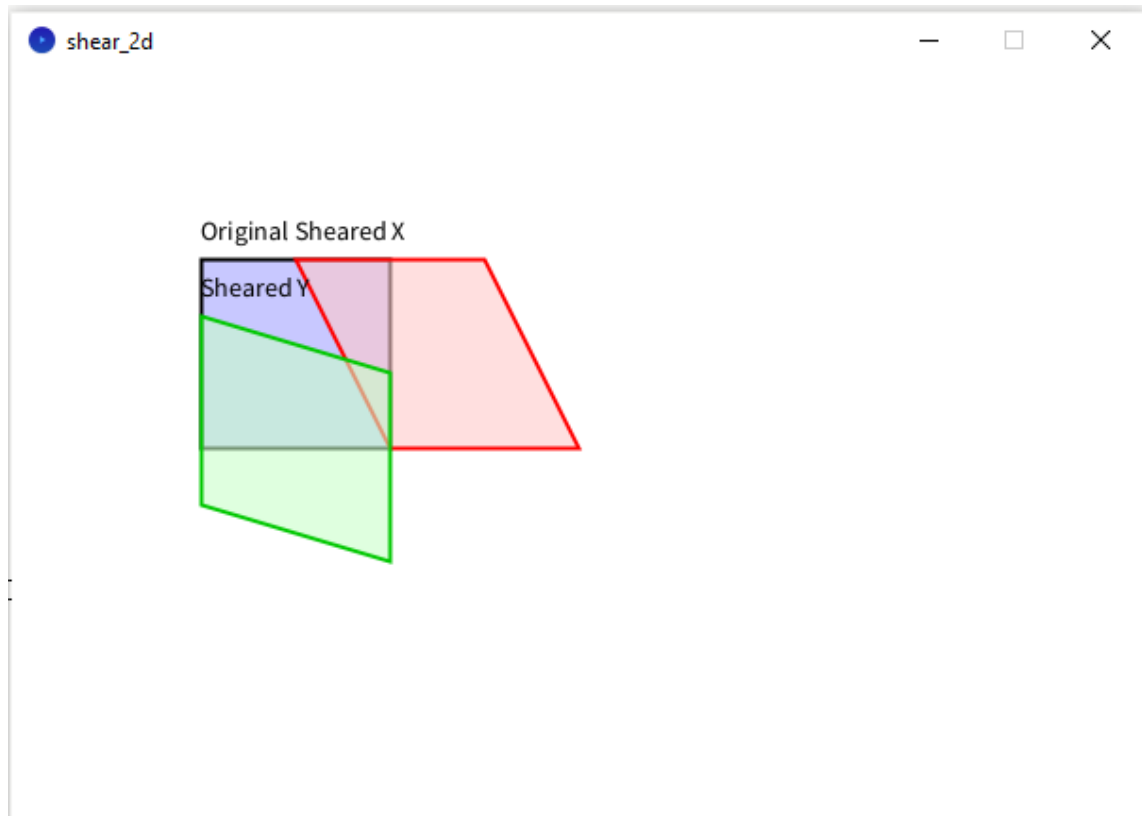
// Sheared in Y
float[][] shearedY = shearY(rectangle, shy);
stroke(0, 200, 0);
fill(200, 255, 200, 150);
drawShape(shearedY);
fill(0);
text("Sheared Y", shearedY[0][0], shearedY[0][1] - 10);
}

void drawShape(float[][] points) {
  beginShape();
  for (int i = 0; i < points.length; i++) {
    vertex(points[i][0], points[i][1]);
  }
  endShape(CLOSE);
}

float[][] shearX(float[][] points, float shx) {
  float[][] result = new float[points.length][2];
  for (int i = 0; i < points.length; i++) {
    float x = points[i][0];
    float y = points[i][1];
    result[i][0] = x + shx * y;
    result[i][1] = y;
  }
  return result;
}

float[][] shearY(float[][] points, float shy) {
  float[][] result = new float[points.length][2];
  for (int i = 0; i < points.length; i++) {
    float x = points[i][0];
    float y = points[i][1];
    result[i][0] = x;
    result[i][1] = y + shy * x;
  }
  return result;
}

```



4.5 Soal Latihan

- Diketahui sebuah bidang segiempat dengan koordinat A(3,1), B(10,1), C(3,5) dan D(10,5).
 - Tentukan koordinat baru dari bidang tersebut dengan melakukan translasi dengan faktor translasi (4,3).
 - Lakukan penskalaan dengan faktor skala (3,2).
 - Dari hasil soal b, lakukan rotasi terhadap titik pusat (A) dengan sudut rotasi 30° .
- Diketahui sebuah objek dengan pasangan koordinat {(4,1), (5,2), (4,3)}.
 - Refleksikan pada cermin yang terletak pada sumbu x
 - Refleksikan pada garis $y=-x$
- Diketahui sebuah bidang segi empat dengan koordinat A(3,1), B(10,1), C(3,5) dan D(10,5).
 - Transformasi shear dengan nilai $shx = 2$
 - Transformasi shear dengan nilai $shy = 2$

Daftar Pustaka

- Andono, Pulung Nurtantio., Sutojo, T. 2016. Konsep Grafika Komputer. Yogyakarta: CV Andi Offset.
- David F. Rogers, Alan J. Adams, Mathematical Elements for Computer Graphics (2nd edition), McGraw-Hill, 1989.
- Edward Angel. 2005. *Interactive Computer Graphics: A Top-Down Approach with OpenGL 2nd*. Addison Wesley.
- Grafika Komputer untuk Mahasiswa dan Umum. Universitas Negeri Malang.
- John F. Hughes, Andries Van Dam, Morgan Mcguire, David F. Sklar, James D. Foley, Steven K. Feiner, Kurt Akeley. 2014. *Computer Graphics: Principles and Practice (3rd edition)*. Addison-Wesley.



6. Klawonn, Frank. 2012. *Introduction to Computer Graphics Using Java 2D and 3D 2nd Edition*. Springer-Verlag: London.
7. Maliki, Irfan. 2006. *Grafika Komputer*. Jakarta Selatan: Mediatek.
8. Siahaan, Vivian., Sianipar, Rismon H., *Java untuk Grafika Komputer dan Animasi*, SPARTA, 2018.